

ROS2K v6.2 Workshop — Student Handout

Dieses Handout begleitet dich durch die 5 Module. Es enthält die Aufgaben, die Kommandos und Platz für deine Notizen. Die Antworten hat nur der Lecturer — du sollst sie selbst entdecken.

Du brauchst:

- Laptop mit NVIDIA GPU + Ollama (`qwen2.5-coder:3b`)
- R2K-HSL -Repo geclont, `./install.sh` durchgelaufen
- `cheatpage.md` (Befehle, KPIs, Szenarien — deine Referenz)
- `part1_boot_ramp.pdf` + `part2_running_system.pdf` (Architektur-Diagramme)
- `rqt_graph_mockup.png` (ROS2 Node Graph)

Alle Kommandos laufen in `~/R2K-HSL/core` .

Glossar — Fachbegriffe

Diese Begriffe tauchen in den Experimenten und Diagrammen auf. Wenn du unsicher bist, schau hier nach.

Begriff	Erklärung	Wo im Workshop
deque	Double-ended queue. Ein Sliding-Window mit fester Länge — alte Einträge fallen hinten raus, neue kommen vorne rein. <code>deque(maxlen=300)</code> = letzte 300 World-States (30s bei 10Hz).	<code>score_node.py</code> , Modul 1
OLS (Ordinary Least Squares)	Lineare Regression. Legt eine Gerade durch Datenpunkte, indem der quadrierte Abstand minimiert wird. Berechnet den Momentum-Trend (rising/falling/stable) aus dem Score-deque.	<code>score_node.py</code> , Modul 1

Begriff	Erklärung	Wo im Workshop
Staleness	Die Verzögerung zwischen Messung (Tracker liest <code>/gazebo/model_states</code>) und Wirkung (Bridge bewegt den Bot). Aktuell ~800ms. Die KI entscheidet immer auf Basis einer leicht veralteten Welt.	Modul 2
min_ents	Die reduzierte Welt, die die KI sieht: Entity-Namen + X/Y-Koordinaten gerundet auf 0.1m. Keine Velocity, kein <code>match_state</code> , kein <code>tactical_score</code> .	<code>r2k_evaluator.py:88</code> , Modul 2
Fragment	Eine Textdatei in <code>strategy/fragments/</code> mit einem Teil des System-Prompts: Regeln, Beispiele oder Persona. Wird von <code>setup_r2k.py</code> beim Boot assembliert.	Modul 4
Oracle / Expert	Menschgeschriebene Analyse-Texte pro Szenario. Oracle = strategisch (was taktisch passieren soll). Expert = technisch (was die KI ausgeben soll). Wird NICHT der KI gezeigt — für menschlichen Vergleich mit <code>--explain</code> -Output.	Modul 2, Modul 5
RPC (Remote Procedure Call)	Ein Nachrichtenformat, bei dem der Sender einen Befehl als JSON-String in einer ROS2-Nachricht serialisiert. Der K1 nutzt <code>booster_msgs/RpcReqMsg</code> mit <code>api_id 2001</code> (Move) oder <code>2000</code> (Failsafe).	Modul 3
num_predict	Ollama-API-Parameter, der wie viele Tokens das Modell generieren darf . <code>--no-explain</code> → 150 (nur assignments). <code>--explain</code> → 600 (analysis + oracle + assignments). Wenn die Antwort das Limit überschreitet → JSON wird abgeschnitten → <code>fast_parse</code> schlägt fehl → kein <code>current_strategy.json</code> → totes blaues Team.	<code>r2k_evaluator.py:111</code> , Modul 1+4
Twist	Standard ROS2-Nachricht für Geschwindigkeitskommandos (<code>geometry_msgs/Twist</code>): <code>linear.x</code> = Vorwärtsgeschwindigkeit, <code>angular.z</code> =	<code>ollama_sandbox_bridge.py</code> , Modul 3

Begriff	Erklärung	Wo im Workshop
	Drehgeschwindigkeit. Sim-Bots und Yahboom nutzen Twist, der K1 nicht.	
tmpfs	Dateisystem im RAM. <code>shared_state/</code> liegt auf tmpfs für atomare Schreib-/Lesezugriffe ohne Disk-I/O. <code>os.replace()</code> ist POSIX-atomar auf demselben Dateisystem.	<code>state_aggregator.py</code> , Modul 2
R2K_RUN_ID	Env-Var, von <code>launch_r2k.sh</code> gesetzt: <code>{scenario}_{strategy}_{timestamp}</code> . Korrelations-Schlüssel für Trace-Dateien (<code>llm_trace_{run_id}.jsonl</code> , <code>world_trace_{run_id}.jsonl</code>).	<code>launch_r2k.sh:87</code> , Modul 2
Composite Score	Gewichtung: $0.4 \times \text{Tor-Diff} + 0.3 \times \text{Tactical-Score} + 0.2 \times \text{Ballbesitz} + 0.1 \times \text{Latenz-Faktor}$. Ein einziger Wert zum Vergleich von Experimenten. ≥ 0.5 ist ein gutes Match.	<code>analyze_trace.py</code> , Modul 1+4
Cold-Boot-Race	Beim ersten Match nach <code>ollama serve</code> lädt Ollama das Modell in VRAM (30-40s). Schließt du in dieser Zeit das Fenster → LLM-Prozess stirbt → kein Strategy-File → totes blaues Team. Zweites Match klappt (Modell ist warm).	Warm-up-Box oben, Modul 1
PID-Controller	Proportional-Integral-Derivative-Regler im Bridge. Berechnet aus Distanz + Winkel zum Target die Geschwindigkeiten <code>lin_x</code> und <code>ang_z</code> . Ist nicht KI-gesteuert — die KI liefert nur den Zielpunkt, der PID fährt dorthin.	<code>ollama_sandbox_bridge.py:188 190</code> , Modul 1+2

Warm up Ollama JETZT

Bevor wir starten: Ollama muss das Modell in VRAM laden. Beim ersten Match nach `ollama serve` dauert das 30-40 Sekunden. Wenn du in dieser Zeit das Fenster schließt, stirbt das blaue Team (kein Strategy-File wird geschrieben).

```
curl -s http://127.0.0.1:11434/api/generate \
  -d '{"model":"qwen2.5-coder:3b","prompt":"hi","stream":false}' > /dev/null
```

```
nvidia-smi # sollte ~2-4GB VRAM zeigen
```

Das war's — jetzt ist das Modell warm. Alle folgenden Matches starten in ~4s.

Wie man die Architektur-Diagramme liest

Du hast zwei Diagramme (A4, Seite-an-Seite legen):

- **Part 1 — Boot & Ramp Phase** (links): Was passiert beim `./launch_r2k.sh` Aufruf. CLI-Flags → `setup_r2k.py` assembelt Fragmente → Ollama-Check → Gazebo-Start → Node-Ignition. Unten stehen die Marks ①-⑤.
- **Part 2 — Running System** (rechts): Was läuft im Steady State. Gazebo → Tracker → Engine-Nodes → Aggregator → Evaluator ↔ Ollama → Bridge → Bots. Oben stehen dieselben Marks ①-⑤.

Die ①-⑤ Marks zeigen, wo Part 1 in Part 2 übergeht:

① Gazebo läuft · ② Bots gespawnt · ③ Ollama warm · ④ `system_prompt.txt` ready · ⑤ Relay+Scenario kopiert.

Leg beide Diagramme nebeneinander: Part 1 rechts unten → Part 2 links oben.

Module 1 — Scoring-Ökosystem (40 min)

Experiment 1: Stack verifizieren (5 min)

```
nvidia-smi # Ollama sollte ~2-4GB VRAM zeigen
curl -s http://127.0.0.1:11434/api/tags # Ollama läuft?
cd ~/R2K-HSL/core
./launch_r2k.sh --headless --duration 30 --scenario 2vs2_default --relay
only_sim_bots
```

Probleme?

- "Ollama not found" → `nohup ollama serve > /dev/null 2>&1 &`
- "Model not pulled" → `ollama pull qwen2.5-coder:3b`

Experiment 2: Live Match mit Visualizer (10 min)

```
./launch_r2k.sh --scenario 3vs3_attack_center --relay only_sim_bots
```

Worauf achten:

- Momentum-Panel: Bewegt sich die Linie? Korreliert sie mit Toren?
- Referee-Zeilen: Erscheinen Set-Pieces? Ball-out → Kick-in?
- HUD: Blue X : Y Red — aktualisiert sich der Score?
- Goalie: Bewegt er sich? (Erwartet: meistens still — bekanntes Problem)

Experiment 3: Sample-Count A/B (10 min)

```
./tools/run_experiment.sh A baseline 120 3vs3_attack_center strat_default --no-explain
python3 tools/analyze_trace.py --run-id <ID>
```

A = Experiment-Name (Baseline). Output: results/A_r1_summary.txt , results/A_r2_summary.txt , results/A_r3_summary.txt . Siehe cheatpage.md §2.

Deine KPIs:

KPI	Run 1	Run 2	Run 3	Mittelwert
goals_for_blue	—	—	—	—
goals_for_red	—	—	—	—
cluster_pct	—	—	—	—
oob_pct	—	—	—	—
latency_p50	—	—	—	—
composite_score	—	—	—	—

Persönliche Beobachtungen / Ideen für eigene Experimente:

- ---
- ---

Experiment 4: 10 Fehler in 10 Minuten (10 min)

```
git show 0566c11:core/src/ros2k_knowledge/R0S2K_GEM_FAQ.md | head -200
```

Finde 5 faktische Fehler im alten FAQ-Text, die nicht mehr stimmen. Tipp: Vergleiche mit dem echten Code (`tracker_node.py` , `launch_r2k.sh` , `ollama_sandbox_bridge.py`).

Gefundene Fehler:

1.

2.

3.

4.

5.

Siehe `cheatpage.md` §3 für alle 14 KPI-Definitionen.

opencode ausprobieren (Modul 1)

How does the referee detect a foul? Show me the threshold values.

What does `score_node.py` momentum calculation do? Explain deque and OLS.

Run a 60s headless match with scenario `3vs3_attack_center` and show me the KPIs.

The goalie is not moving. Check `ollama_sandbox_bridge.py` for how it handles the goalie bot. Show me the PID control section.

Key Take-Aways Module 1

- **Scoring ist ein 3-Node-System:** Referee (Fouls/Set-Pieces) → Score (Tactical + Momentum OLS über 30s) → Reward (1Hz, -10..+10). Alles fließt durch flat JSON, nicht durch ROS-Topics.

- **1 Sample schlägt 6 Samples:** Das 3B-Modell kopiert ein Muster, es lernt nicht aus Diversität. Mehr Samples verdünnen den Fokus und erhöhen die Latenz.
 - **--explain senkt OOB auf 1.9%, kostet aber +44% Latenz** (600 statt 150 Tokens). Nutze es für Debugging, nicht für Produktion.
 - **Goalie idle ~95% ist strukturell:** Der Bridge-PID verfolgt ein ball-Y Setpoint, das ~800ms alt und jittery ist. Kein Prompt-Fix — Phase 5.1 (Kalman) ist die Lösung.
 - **KPIs sind deine Wahrheit:** `composite_score` fasst alles zusammen. Ohne `analyze_trace.py` fliegst du blind.
-

Module 2 — World Model (35 min)

Experiment 1: Was sieht die KI? (8 min)

```
python3 tools/dump_prompt.py --scenario 3vs3_attack_center --strategy  
strat_default --no-explain  
cat shared_state/Worldstate.json | python3 -m json.tool
```

Vergleiche: `Worldstate.json` enthält `match_state`, `tactical_score`, alle Entities. Aber `r2k_evaluator.py` reduziert auf `min_ents` (nur X/Y, gerundet auf 0.1m).

Was fehlt der KI?

- -
 -
-
-
-

Siehe `part2_running_system.pdf`: Finde den "mtime poll 20ms" Pfeil zwischen `Worldstate.json` und `r2k_evaluator.py`.

Experiment 2: Staleness messen (7 min)

```
ls logs/llm_trace_*.jsonl logs/world_trace_*.jsonl | head -4  
# Wähle eine Run-ID, schaue die Timestamps an:
```

```
head -1 logs/llm_trace_<ID>.jsonl | python3 -c "import json,sys;
d=json.load(sys.stdin); print(f'LLM call at t={d[\"t\"]:.3f}')"
head -1 logs/world_trace_<ID>.jsonl | python3 -c "import json,sys;
d=json.load(sys.stdin); print(f'World at t={d[\"t\"]:.3f}')"

```

Deine Messung:

- World-Timestamp: _
- LLM-Timestamp: _
- Staleness (Delta): _ ms
- Erwartet: 700-900ms (B-Studie p50: 742-827ms)

Experiment 3: Oracle/Expert-Vergleich (12 min)

```
cat scenario/3vs3_attack_center/analysis.md
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_attack_center --
relay only_sim_bots --explain

```

Nach dem Match: LLM-Reasoning im `llm_trace` ansehen (`raw_response` enthält `analysis` + `oracle` Keys). Vergleiche mit der menschlichen Oracle.

Diskussion:

- Stimmt das LLM-Reasoning mit der Oracle überein?
- Wenn KPIs gut sind, aber das Reasoning Unsinn ist — ist das eine echte Verbesserung?
- Wenn KPIs schlecht sind, aber das Reasoning sinnvoll ist — schlechter Prompt oder Varianz?

Siehe `cheatpage.md` §4: Szenario-Liste mit Oracle + KPI-Targets.

opcode ausprobieren (Modul 2)

Read the latest `world_trace` file and tell me: how many frames, what was the ball position at frame 100, and what was the `match_state.status` distribution?

What does the LLM actually see? Show me the `min_ents` stripping in `r2k_evaluator.py` and compare it to the full `Worldstate.json`.

Why is the LLM producing empty JSON? Check the last `llm_trace` file for parse errors.

Key Take-Aways Module 2

- **Ground Truth = /gazebo/model_states only.** Kein /odom, kein TF2, kein IMU. Der Tracker extrahiert nur position.x und position.y — kein Yaw, keine Quaternionen. 2D reicht für Soccer-Taktik.
- **Die KI sieht eine reduzierte Welt:** min_ents = X/Y gerundet auf 0.1m. Kein match_state, kein tactical_score, keine Velocity. Was die KI nicht sieht, kann sie nicht nutzen.
- **Staleness ~800ms ist real:** Ball bei 2 m/s → Ball ist 1.6m weg von dort, wo die KI denkt. Das ist warum der Goalie stillsteht — er rennt zur Position von damals, nicht von jetzt.
- **Trace-Logging = Observability:** llm_trace (pro LLM-Call) + world_trace (pro 10Hz-Tick), korreliert via R2K_RUN_ID. 14 KPIs offline berechenbar. Ohne Traces kein Debugging.
- **Oracle/Expert = menschlicher Maßstab:** KPIs sagen OB die KI gut spielt. Oracle/Expert sagt WARUM. Du brauchst beides.

Module 3 — K1-Anbindung & Thresholds (50 min)

Experiment 1: Relay-Inspektion (8 min)

```
cat relay/only_sim_bots.json | python3 -m json.tool
cat relay/hardware_mirror.json | python3 -m json.tool
```

Unterschied:

- only_sim_bots: _____
- hardware_mirror: _____

| Siehe rqt_graph_mockup.png: Finde die /cmd_vel und /LocoApiTopicReq Topics.

Experiment 2: Hysterese-Demo (12 min)

```
python3 -m pytest tests/test_foul_detection.py -v -s
```

Patche HYSTERESIS_FRAMES=1 in referee_node.py (oder im Test), re-run.

Beobachte: Mehr Fouls erkannt (weniger Filterung = empfindlicher).

Was macht Hysterese?

•

Experiment 3: K1 mit opencode erkunden (10 min)

How is the Booster K1 controlled? Show me the booster_msgs publishing code in ollama_sandbox_bridge.py. What are API codes 2000 and 2001?

K1 API-Codes:

- 2001 = ____
- 2000 = ____

Experiment 4: Korridor-Walk (8 min)

```
python3 -c "  
import json, matplotlib.pyplot as plt  
records = [json.loads(l) for l in open('logs/world_trace_<ID>.jsonl')]  
scores = [r.get('tactical_score', {}).get('momentum_30s', 0) for r in  
records]  
times = [r['t'] - records[0]['t'] for r in records]  
plt.plot(times, scores)  
plt.axhspan(-0.5, 0.5, alpha=0.2, color='gray', label='stable corridor')  
plt.axhspan(0.5, 2.0, alpha=0.2, color='green', label='rising')  
plt.axhspan(-2.0, -0.5, alpha=0.2, color='red', label='falling')  
plt.xlabel('Time (s)'); plt.ylabel('Momentum 30s'); plt.legend()  
plt.savefig('momentum_corridor.png')  
"
```

Threshold-Taxonomie

Konzept	Erklärung	Wo in ROS2K?
Threshold	Einzelwert-Vergleich (if dist < 0.3)	Referee: Distanz/Geschwindigkeit
Hysteresese	Musste N Frames persistieren, verhindert Flackern	Referee: HYSTERESIS_FRAMES=3
Korridor	Akzeptabler Bereich (±0.5 = "stable")	Momentum-Trend-Klassifikation
Probability	Stochastische Konfidenz	Nicht verwendet

opencode ausprobieren (Modul 3)

How is the Booster K1 controlled? Check the edge hardware power-file.
Show me the booster_msgs publishing code in ollama_sandbox_bridge.py.
What are API codes 2000 and 2001?

Run a 30s headless match with `--relay hardware_mirror` and compare the relay JSON mapping to what topics actually show up.

Key Take-Aways Module 3

- **K1 nutzt ROS2, aber nicht Standard-Twist.** Custom `booster_msgs/RpcReqMsg` mit JSON-serialisiertem RPC: `api_id 2001 = Move ( vx, vy, vyaw ),`
`api_id 2000 = Failsafe.` Topic-Name kommt aus `relay/hardware_mirror.json`.
- **Keine OOP HALs.** Die Bridge nutzt dynamische Thread-Closures (`def task`), keine `BaseBotDriver`-Vererbung. Das ist ein bewusstes Architektur-Axiom.
- **K1-Freeze ist sim-only.** Der Referee friert via `cmd_vel` Twist-zero ein, aber der K1 ignoriert `cmd_vel`. Set-Piece-Freeze funktioniert nur in Gazebo.
- **Threshold + Hysterese = ROS2K heute.** Threshold = Einzelwert-Vergleich, Hysterese = N-Frames-Persistenz gegen Flackern. Korridor = Momentum-Trend. Probability = nicht verwendet.
- **Relay-JSON ist die Hardware-Brücke.** `only_sim_bots` = alles virtuell.
`hardware_mirror` = maps Bots zu echten Topics. Ohne Relay-Datei → Fallback ohne Hardware.

Module 4 — Utils & Fragments (35 min)

Die Iterations-Schleife

Fragment editieren → `dump_prompt.py` (verifizieren) → Match laufen lassen
→ `analyze_trace.py` (KPIs) → mit Baseline vergleichen → wenn besser:
committen

Experiment 1: KPIs lesen (8 min)

```
python3 tools/analyze_trace.py --run-id <ID aus Module 1>
```

Schreibe die wichtigsten KPIs auf:

- `composite_score : _`

- oob_pct:_
- cluster_pct:_
- goalie_idle_pct:_
- latency_p50:_

Experiment 2: Fragment-Surgery (10 min)

```
python3 tools/dump_prompt.py --scenario 3vs3_attack_center --strategy
strat_default --no-explain
# Editiere rules_core.txt – füge eine Regel hinzu (z.B. "ALWAYS pass to
closest bot to goal")
python3 tools/dump_prompt.py --scenario 3vs3_attack_center --strategy
strat_default --no-explain
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_attack_center --
relay only_sim_bots
python3 tools/analyze_trace.py --run-id <ID>
```

Vergleich:

KPI	Baseline (Module 1)	Nach Edit	Delta
composite_score	—	—	—
oob_pct	—	—	—
cluster_pct	—	—	—
goals_for_blue	—	—	—

Experiment 3: opencode Fragment-Edit + Experiment (7 min)

Edit rules_core.txt to add a rule: "ALWAYS pass to the bot closest to the goal" if it's not already there. Then run dump_prompt.py to verify the prompt includes the new rule.

Run a 60s headless match with scenario 3vs3_attack_center and --no-explain, then run analyze_trace.py on the result and show me the KPIs. Compare goals_for_blue and cluster_pct to the baseline run from Module 1.

Plan-Modus (Tab-Taste):

```
<Tab> "I want to add a new rule to rules_core.txt that prevents bots from clustering. Show me a plan before making changes." <Tab> "Go ahead."
```

Siehe `cheatpage.md` §1 (Launch-Flags), §2 (Test-Kommandos), §5 (Quick Recipes).

Key Take-Aways Module 4

- **Fragment-Assembly passiert beim Boot, nicht zur Laufzeit.** `setup_r2k.py` liest `fragments/` → schreibt `system_prompt.txt`. Editiere immer die Fragmente, nie `system_prompt.txt` (wird bei jedem Boot überschrieben).
- **Override-Logik:** Strategy-Fragmente (`rules_aggro.txt`) ersetzen Mode-Fragmente (`rules_3vs3.txt`), wenn sie existieren. Samples ebenso.
- **`dump_prompt.py` ist dein Pre-Flight-Check.** Zeigt den assemblierten Prompt ohne ROS/Ollama. immer ausführen bevor du ein Match startest.
- **Die Iterations-Schleife ist manuell:** Fragment editieren → `dump_prompt` → Match → `analyze_trace` → KPIs vergleichen → wenn besser, committen. Kein automatischer Optimierer — Trial-and-Error mit `pytest` als Regressionsschutz.
- **`run_experiment.sh` A baseline:** A = Name (Baseline), `baseline` = keine Fragment-Swap. B1 - B7b = Varianten mit `swap_fragments.sh`.

Module 5 — Forschungs-Roadmap (45 min)

Experiment 1: Make it your own (10 min)

```
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_attack_center --  
relay only_sim_bots  
python3 tools/analyze_trace.py --run-id <ID>
```

Das ist dein persönlicher Datensatz vom Workshop-Tag.

Deine KPIs:

- `composite_score`: _
- `tactical_score_avg`: _
- `ball_possession_blue_pct`: _
- `role_diversity`: _
- `status_distribution`: _

Persönliche Beobachtungen / Ideen für eigene Experimente:

- ---
- ---

Experiment 2: Oracle/Expert mit --explain (10 min)

```
cat scenario/3vs3_attack_center/analysis.md
./launch_r2k.sh --headless --duration 60 --scenario 3vs3_attack_center --
relay only_sim_bots --explain
```

Vergleiche das LLM-Reasoning (analysis + oracle im llm_trace) mit der menschlichen Oracle aus analysis.md .

Hat die KI richtig gedacht?

- ---
- ---

Phase 5 — Forschungsrichtungen

Richtung	Was	Status
5.1 Kalman-Filter	Noise filtern, Velocity schätzen. Goalie-Idle-Fix.	Geplant
5.2 Predictive World Model	Welt ~800ms vorhersagen → Latenz kompensieren	Geplant
5.3+5.4 Watchdog + Failsafe	Predicted vs. Actual vergleichen → Rule-based Fallback	Geplant
5.5 Sim-to-Real	Auf K1/Yahboom-Hardware testen	Geplant
5.10 5vs5 Scale-Up	5 Bote, mehr Rollen, größere Prompts	Geplant
5.11 LLM-Output-Quality	Automated LLM-as-judge für Reasoning-Quality	Geplant

Welche Richtung interessiert dich für ein Praktikum / Studienprojekt?

- ---

•

opencode ausprobieren (Modul 5)

```
Run a 60s headless match with scenario 3vs3_attack_center and --explain,
then extract the LLM's analysis and oracle fields from the llm_trace and
compare them to the oracle text in scenario/3vs3_attack_center/analysis.md.
```

```
Explain the Phase 5.1 Kalman filter plan. Where in tracker_node.py would
it be implemented? Show me the current code that would need to change.
```

Key Take-Aways Module 5

- **Phase 5 ist Forschung, nicht Implementiert.** Alles hier ist geplant.
Sage "wir planen", nicht "wir haben". Keine Features als existent präsentieren.
- **Kalman-Filter (5.1) ist der Enabler.** Filtert Noise, schätzt Velocity, behebt Goalie-Idle. Ohne 5.1 kein 5.2 (Predictive World Model).
- **Predictive World Model (5.2) kompensiert Latenz.** Vorhersage um ~800ms
→ KI entscheidet für die Welt, wie sie SEIN WIRD, nicht wie sie WAR.
Reduziert effektive Staleness auf nahe Null.
- **Watchdog + Failsafe (5.3+5.4) = Sicherheit.** Predicted vs. Actual vergleichen → bei Divergenz → rule-based Fallback. System hängt nie, produziert nie gefährliche Kommandos.
- **Sim-to-Real (5.5) ist der Endtest.** --relay hardware_mirror auf K1/Yahboom. Sim-KPIs vs. Feld-KPIs vergleichen. K1-Freeze-Limit beachten.
- **5vs5 (5.10) ist offen:** Schafft das 3B-Modell 5-Bot-Koordination oder brauchen wir 7B? Mehr Rollen, größere Prompts, höhere Latenz.

Wenn etwas schiefgeht

Problem	Fix
Ollama nicht erreichbar	<code>pkill -9 -f "ollama runner"; pkill -9 -f "ollama serve"; sleep 2; nohup ollama serve > /dev/null 2>&1 &; sleep 3; nvidia-smi</code>
Blaues Team bewegt sich nicht (dead blue)	<code>ls logs/llm_trace_*_<run_id>.jsonl</code> — wenn Datei fehlt: Cold-Boot-Race. Match neu starten (Modell ist jetzt warm).

Problem	Fix
Gazebo startet nicht	<code>ros2 run r2k_world_model tracker</code> in anderem Terminal prüfen. Docker: <code>docker ps</code> — Container läuft?
<code>parse_error_rate</code> hoch	<code>num_predict</code> zu niedrig? Versuche <code>--explain</code> (600 Tokens statt 150). Siehe <code>cheatpage.md §3</code> .

Wo finde ich was?

Was	Wo
Launch-Flags (<code>--scenario</code> , <code>--explain</code> , etc.)	<code>cheatpage.md §1</code>
Test-Kommandos (<code>pytest</code>)	<code>cheatpage.md §2</code>
14 KPI-Definitionen	<code>cheatpage.md §3</code>
10 Szenarien + Oracle + KPI-Targets	<code>cheatpage.md §4</code>
Quick-Test-Rezepte (Smoke test, etc.)	<code>cheatpage.md §5</code>
Dateipfade (wo liegen die Files?)	<code>cheatpage.md §6</code>
Architektur-Übersicht (Boot)	<code>part1_boot_ramp.pdf</code>
Architektur-Übersicht (Runtime)	<code>part2_running_system.pdf</code>
ROS2 Node Graph	<code>rqt_graph_mockup.png</code>
Referee-Regelwerk (alle Entscheidungen)	<code>core/docs/referee_rulebook.md</code>
Optimierungs-Spec (Phasen 0-5)	<code>core/docs/optimization_spec_v6.2.md</code>
opencode starten	<code>cd ~/R2K-HSL/core && opencode</code>